

Badminton Player Tracking and Tactical Analysis System

Dang Huu Thang
24146078

June 28, 2026

1 Introduction

This project applies computer vision techniques to badminton player tracking and tactical analysis, supporting a more data-driven approach to training and match evaluation.

2 Methodology

2.1 Problem Statement

Analyzing player movement and tactics in badminton matches from conventional video footage faces two major challenges: perspective distortion caused by camera viewing angles and the inaccurate process of manually determining player positions. To address these issues, this project proposes a technical solution that combines traditional image processing techniques to normalize the court geometry with Deep Learning methods for automatic player localization. The objective is to develop a software system capable of automatically extracting player movement trajectories and visualizing them as quantitative tactical analysis data, and optimizing hardware resource utilization to ensure efficient and practical deployment.

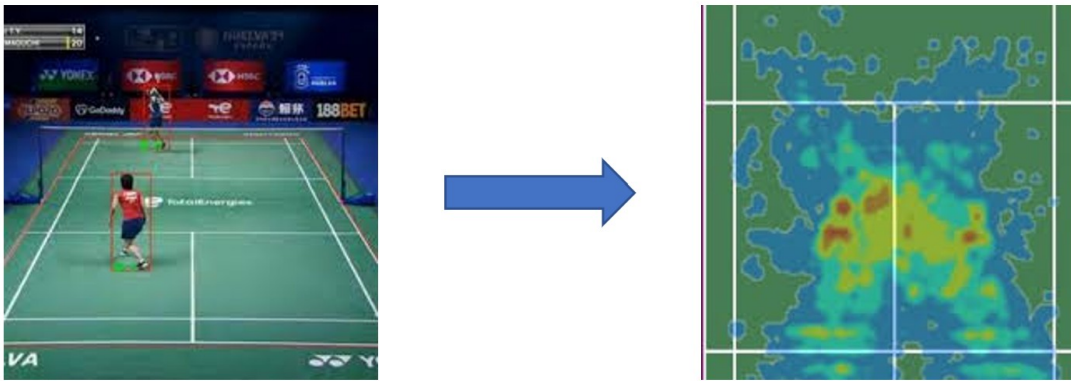


Figure 1: Heatmap analysis

2.2 Court detection

Directly detecting the white court lines using techniques such as Hough Lines, Convex Hull, or contour extraction often fails due to noise introduced by lighting conditions, player clothing, and

advertising boards. On the other hand, employing deep learning models such as YOLO for this task would result in unnecessary computational overhead and hardware resource consumption.

To address these limitations, a hierarchical detection strategy is proposed. First, the system exploits the green court surface, which occupies a large area, remains visually stable, and can be easily segmented using the HSV color space. After extracting the green-court mask, it is used as a spatial constraint. Instead of blindly connecting white line segments, a Bitwise AND operation is applied to restrict the search region, allowing only white pixels located within the court area to be retained. As a result, most environmental noise is effectively eliminated.



Figure 2: Green and white masks

Finally, to overcome the issue of fragmented court lines causing geometric distortions during point connection, the system performs automatic coordinate normalization by averaging the Y-coordinates of detected vertices. This process constrains the detected corners to conform to the standard court geometry, ensuring a robust and consistent structural representation of the badminton court.

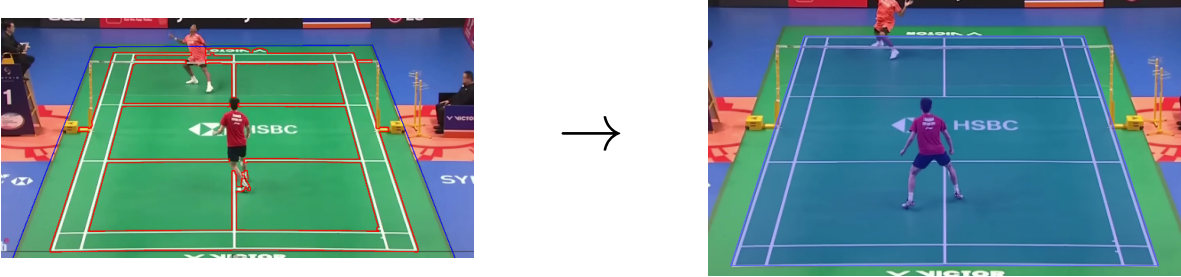


Figure 3: Court extraction and tactical visualization process

2.3 Feet Tracker

To determine the player’s foot position in each frame, the system does not use traditional target tracking methods (such as Optical Flow or Background Subtraction) because they are easily interfered with by shadows or racket movements. Instead, the project applies a Convolutional Neural Network (CNN) through the ResNet18 architecture, which is well tuned to perform the regression task.

2.3.1 Data Preprocessing and Synchronization

To ensure the model learns geometric features instead of being misled by clothing colors, all input images must go through a strict preprocessing pipeline including: resizing by compressing the frame to a fixed resolution of 416x416 to reduce computational costs; decolorizing and smoothing by converting the image to grayscale and applying a 5x5 Gaussian Blur kernel to blur unnecessary noise details, helping the neural network focus on the player’s shape; and finally Tensor normalization by converting

the image data into a Tensor format and normalizing it according to the standard PyTorch mean and standard deviation values before feeding it into the network.

2.3.2 Customizing the ResNet18 Architecture

The original ResNet18 is designed for image classification tasks with thousands of output classes. To apply it to the localization task, the fully connected layer at the end of the model was removed and replaced with a new linear layer: `nn.Linear(num_fts, 2)`. This modification forces the model to output a vector with exactly 2 continuous values, corresponding to the predicted x and y coordinates of the feet in the 416x416 image space. The Adam optimization algorithm and Mean Squared Error (MSE) loss function are used to minimize the Euclidean distance between the predicted and actual coordinates.

2.3.3 Inference and Rescaling Process

In the actual execution flow, when a new video frame is input, the AI calculates and returns the (x, y) coordinates on the 416x416 image scale. To display accurately on the original video, the system performs a rescaling operation by calculating the scale factor between the original video size and the 416x416 size, then multiplying the AI-predicted coordinates by this scale factor to return the detected point to the exact physical position on the screen, ready for the perspective projection step.

2.3.4 Results of training process

The training process of the ResNet18 neural network was strictly monitored and recorded via the `loss.history.txt` file.



Figure 4: Loss results of 1600 images

2.4 Geometry Transformation and Heatmap Visualization

After extracting the court boundaries and the player's foot coordinates, the system needs to bring this spatial data into a common reference system on a 2D plane for accurate tactical evaluation.

2.4.1 Court Flattening Algorithm

To eliminate image distortion caused by tilted camera angles, the system utilizes the Perspective Transform. Based on the standard 4 court corner coordinates locked from the image processing module, the system calculates the perspective matrix via the `cv2.getPerspectiveTransform` function, and then

uses this matrix to map and flatten the entire court area into a 2D rectangle with target dimensions. Notably, this perspective matrix is continuously used in combination with the `cv2.perspectiveTransform` function to project the AI-predicted foot coordinates onto the new flat space system, helping to completely eliminate distance errors caused by the lens's effect of the camera.

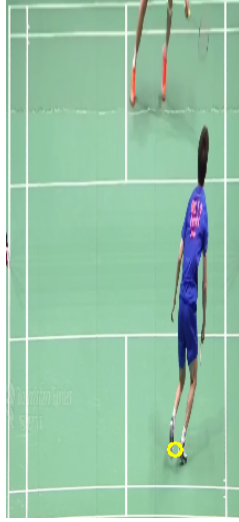


Figure 5: Flattened

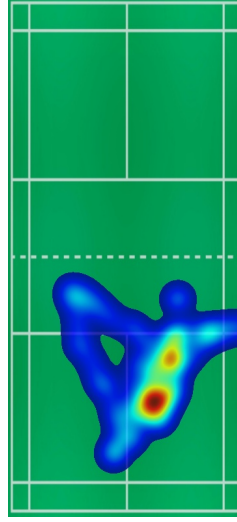


Figure 6: Heatmap

2.4.2 Data Accumulation and Static Heatmap Construction

Instead of drawing overlapping movement trajectories on the original video, the system builds a static heatmap module operating on a high-definition (800px) graphic court background image to provide the most comprehensive analytical view. The flattened foot coordinates continue to be spatially scaled and their values (frequency of appearance) are accumulated into an Accumulation Matrix. When the video analysis process concludes, the system diffuses this density matrix using a Gaussian Blur filter with an extremely large kernel size (91x91), normalizes the floating-point data to an 8-bit system, and applies the `cv2.COLORMAP_JET` color map to create a thermal color spectrum. Finally, through Thresholding techniques for background separation, the heatmap region is sharply blended onto the static court surface using the `cv2.addWeighted` operation, outputting the final tactical map that clearly reflects the player's movement habits. The results is shown in Figure. 6.

3 Self-Assessment

3.1 Positive features of the solution

Rather than relying on increasingly complex algorithms that require high-performance hardware, this project focuses on optimizing the information processing pipeline through a Hierarchical Detection strategy. The key innovation is exploiting the green court surface to filter out noise, enabling the court boundary detection problem to be solved using purely geometric image processing operations (OpenCV) instead of computationally expensive Deep Learning models.

In addition, the development of a custom data labeling tool with a progress save mechanism and the implementation of a geometry normalization algorithm based on coordinate averaging help overcome line discontinuities caused by occlusions. These design choices allow the system to operate reliably on the target video dataset while maintaining low computational requirements, making it suitable for deployment on resource constrained hardware.

3.2 Limitations of Traditional Approaches

Although the proposed system achieves satisfactory performance under controlled conditions, it still faces several limitations when compared with large scale research systems. The color segmentation parameters (HSV thresholds) and binary threshold values are currently hardcoded, making the system sensitive to sudden lighting variations or courts that do not conform to standard color specifications.

Furthermore, the regression-based AI model is not yet capable of handling more challenging scenarios, such as prolonged occlusions or doubles matches, where players' feet may overlap and become difficult to distinguish, the change of the camera angles during cutscenes, etc. These limitations represent inherent challenges of the current approach and could be addressed in future work through the integration of more advanced Deep Learning techniques, such as Pose Estimation combined with object detection frameworks like YOLO.

4 References

1. Bradski, G., & Kaehler, A. (2008). *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media
2. He, K., Zhang, X., Ren, S., & Sun, J. (2016). "Deep Residual Learning for Image Recognition". *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
3. Szeliski, R. (2022). *Computer Vision: Algorithms and Applications* (2nd ed.). Springer.
4. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
5. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
6. Zheng, A. (2015). *Mastering Feature Engineering: Take Your Data Science Projects to the Next Level with Feature Engineering Techniques*. O'Reilly Media